Paiflow API quick start for developers

Build your app's checkout, donations or payment experience while Paiflow handles the money rules. Draw and deploy a flow in Paiflow, then use the API to prepare a payment, have the customer sign it in their wallet, submit it and read the results.

Use **Stellar testnet** for the hackathon. Testnet balances have no real-money value. The API drives an existing deployment; it does not create flows or deploy copies for each customer.

1 Set up your flow and API access

1. Sign in with your team's account at `https://beta.app.paiflow.xyz`.
2. Build a simple **On Receive → Pay** flow. Choose XLM and pay the full amount to a funded testnet recipient. Use Split, Swap or conditions when your app needs them.
3. Connect your team's wallet, sign deployment and wait for **CONFIRMED**.
4. Open the deployment's **API access** panel, create a token and save it immediately. The full token is shown once.
5. Copy the **deployment UUID**. This is different from the flow ID and the `C…` contract address.

Keep the `pfk_…` token in your backend's secret store. Each token belongs to one deployment. Customers sign payments in their own wallets; neither your backend nor Paiflow needs their secret seed. The **Try the sandbox** identity cannot mint tokens or call `/api/v1`; use your team account.

One deployment serves your team's business or group. Customers share its configured recipients and conditions. Subscription flows have one configured subscriber. Design your demo around those shared rules.

2 Prepare a payment

The terminal examples need `curl` and `jq`. Replace the placeholders with your deployment, token and the paying wallet's public address:

```
export PAIFLOW='https://beta.app.paiflow.xyz'
export DEPLOYMENT_ID='YOUR-DEPLOYMENT-UUID'
export PAIFLOW_TOKEN='pfk_YOUR_64_HEX_CHARACTERS'
export FROM='YOUR_CUSTOMER_G_ADDRESS'
export AMOUNT='10000000'
export BASE="$PAIFLOW/api/v1/deployments/$DEPLOYMENT_ID"
```

`amount` is a **positive integer string in stroops of the input asset**: 1 XLM = `"10000000"`, 10 USDC = `"100000000"`. Use `bigint` for arithmetic and strings in JSON. The `from` account supplies the deposit, pays the fee and signs. It must be funded on testnet; recipients of USDC or another issued asset need the corresponding trustline.

```
jq -n --arg from "$FROM" --arg amount "$AMOUNT" \
  '{from: $from, amount: $amount}' > prepare-body.json

curl --fail-with-body -sS -X POST "$BASE/execute" \
  -H "Authorization: Bearer $PAIFLOW_TOKEN" \
  -H 'Content-Type: application/json' \
  --data-binary @prepare-body.json > prepared.json

jq '.data' prepared.json
```

Preparation simulates the payment and returns `data.xdr`, `data.networkPassphrase`, `data.network` and `data.expiresAt`. Check that `network` is `testnet`. The envelope has a **180-second validity window**; have wallet signing ready before preparing.

`execute` supports flows headed by **On Receive**. Schedule, Subscription and HTTP Webhook use their own funding, allowance or callback workflows. A Dev flow with an unset recipient must be configured before execution.

3 Get the customer’s wallet signature

Send the unsigned `data.xdr` and returned `data.networkPassphrase` to your frontend. Ask the customer’s wallet to sign that exact transaction on the supplied network. Send the signed envelope back to your backend.

For the terminal walkthrough, save the wallet’s signed base64 envelope in `signed.xdr`. Sign and submit before `expiresAt`. Adding a signature is sufficient; keep the transaction body unchanged.

Your app’s path is:

Frontend requests payment → backend prepares → customer wallet signs → backend submits → app checks outcome.

Keep the API token in the backend. Display the amount, asset and testnet network before requesting the customer’s signature.

4 Submit and check the outcome

```
jq -n --rawfile signedXdr signed.xdr \
  '{signedXdr: ($signedXdr | gsub("\\s"; ""))}' > submit-body.json

curl --fail-with-body -sS -X POST "$BASE/execute/submit" \
  -H "Authorization: Bearer $PAIFLOW_TOKEN" \
  -H 'Content-Type: application/json' \
  --data-binary @submit-body.json > submitted.json

jq '.data' submitted.json
```

Successful HTTP responses wrap the result in `data`, including `txHash` and `status`:

Status	What your app should do
`SUCCESS`	The transaction is confirmed. Read its events to see whether funds were paid, split, swapped or held.
`PENDING`	Keep the payment pending. Retry submit with the **same signed envelope** to check again.
`FAILED`	Show the returned `data.error` and resolve the cause before attempting another payment.

HTTP 200 alone does not mean payment success: inspect `data.status`. Default submission waits for a bounded period; add `?wait=false` to return without waiting for finality. If submission times out or returns `502`, it may already have broadcast. Preserve and retry the same envelope instead of preparing another payment. Retries use its transaction hash for idempotency.

5 Read payment events

```
TX_HASH=$(jq -er '.data.txHash' submitted.json)
curl --fail-with-body -sS --get "$BASE/events" \
  -H "Authorization: Bearer $PAIFLOW_TOKEN" \
  --data-urlencode "txHash=$TX_HASH" \
  --data-urlencode 'limit=100' | jq '.data'
```

The response contains `items`, `nextCursor` and `hasMore`. Events include `eventId`, `txHash`, `kind`, `topic` and `data`. A swap uses `kind: "PAYOUT"` with `topic: "swap"`; its data includes `assetIn`, `assetOut`, `amountIn` and `amountOut`.

For ongoing polling, save `nextCursor` and pass it as `cursor` on the next request. Fetch immediately while `hasMore` is true, then wait about 30 seconds. Omit `cursor` when it is null. Keep cursors opaque and deduplicate processing by `eventId`. Events are deployment-wide, so associate your app's payment record with its `txHash`. A confirmed deposit can leave money held by a condition; check the expected payout event before marking a recipient paid.

Optional recipients and early release

For a different recipient on each payment, use a Dev flow with an immediate mutable Pay, such as **On Receive → Pay** with **Fill value via API**. Prepare with `POST /payouts` using `{ "from": "G…", "amount": "10000000", "recipient": "G…" }`, then sign and submit to `POST /payouts/submit`. Add `nodeId` when there are multiple deployed Dev Pay nodes. These paths are relative to `$BASE`.

Payout envelopes have a **60-second validity window**. The selected Pay must have zero existing balance and consume all incoming funds; deferred values are set to 100%. Recipient accounts and required trustlines must exist. Unsupported or delayed paths are refused. A `409 CONFLICT` means an earlier payout may still execute: retain its envelope, respect `Retry-After` when supplied and wait for finality or verified expiry. Run payouts sequentially; direct deposits, `execute` and other configuration changes bypass their coordination. See the full payout rules before using a more complex graph.

For a funded time condition that allows early release, call `POST /release-early` with `{ "nodeId": "YOUR-CONDITION-NODE-ID" }` before its deadline. The token authorises Paiflow's relayer to release the before side. Your backend must decide who may request this action. A strict Hold prevents early release; deposits in the same condition share its balance and deadline.

Troubleshooting and reference

Errors use `{ "error": { "code": "…", "message": "…", "fields": { … } } }`. Keep the response's `x-request-id` when reporting a problem.

HTTP	Action
`401`	Check that the token is active and belongs to this deployment.
`402`	Fund the paying account on testnet.
`404`	Check the deployment UUID and confirmation status.
`409`	Wait for the earlier payout to resolve.
`422`	Read the message and fields; check amounts, trigger type, recipients and trustlines.
`429`	Back off using `Retry-After`.
`502`	For submission, retry the same signed envelope; avoid duplicate payments.

Per-token limits per minute: execute and submit **30 each**, events **120**, payout preparation **10**, payout submission **30**, early release **10**.

- API reference
- Hackathon API and flow context
- OpenAPI specification
- Postman collection — set `baseUrl` to `https://beta.app.paiflow.xyz` and fill in your deployment, token and paying address.

Branch baseline: `origin/hackathon` at `1b89b9f2d9565930b20268c9a8db76f3a36b081e`.